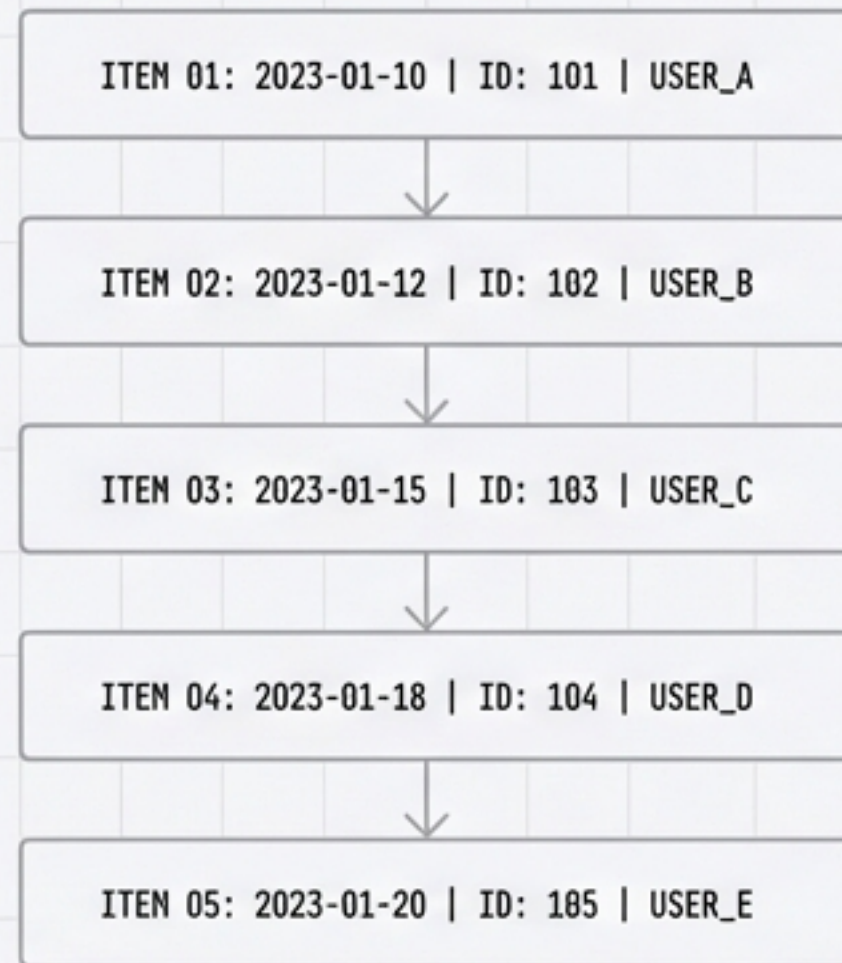


THE NEWS FEED IS A DERIVED VIEW, NOT A SOURCE OF TRUTH

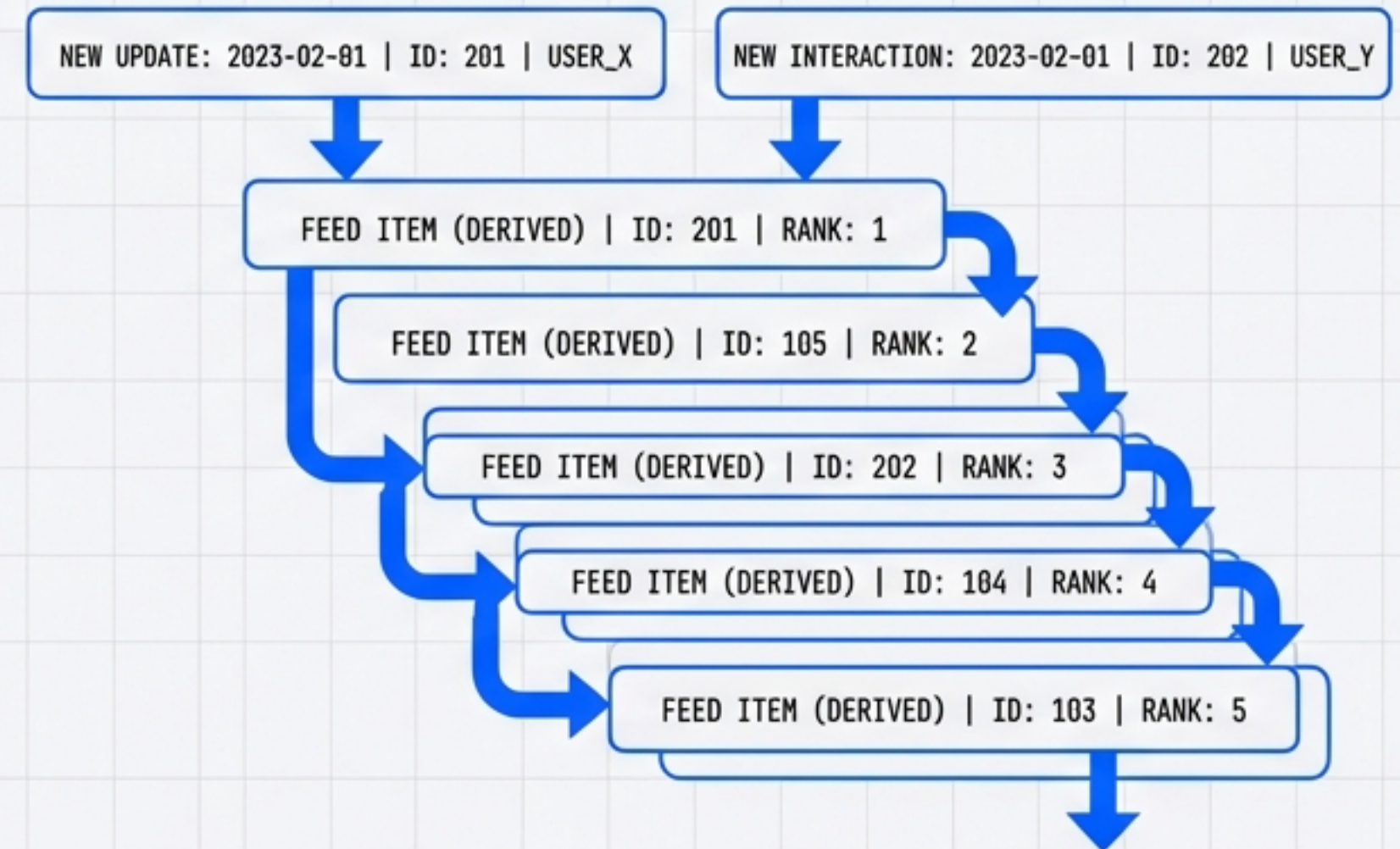
A News Feed answers the question: "What is new and relevant to this user right now?"
It is a continuously updating, personalized, ranked, and filtered stream.

STATIC LIST



FIXED COLLECTION OF DATA (RIGID)

DYNAMIC FEED



CONTINUOUS STREAM (FLOWING & PERSONALIZED)

KEY CONCEPT

AVAILABILITY > CORRECTNESS

The feed is a materialized projection. If a feed is slightly stale, missing a post, or ordered differently, the system functions. However, if the feed fails to load, the app feels broken. Downtime hurts user perception significantly more than data staleness.

CORE FUNCTIONAL GOALS AND NON-FUNCTIONAL CONSTRAINTS

FUNCTIONAL REQUIREMENTS

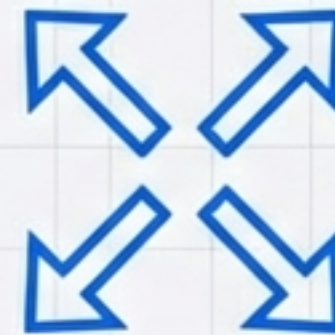


NON-FUNCTIONAL CONSTRAINTS

LOW LATENCY



Use Skeleton Loading to manage user perception.



SCALABILITY

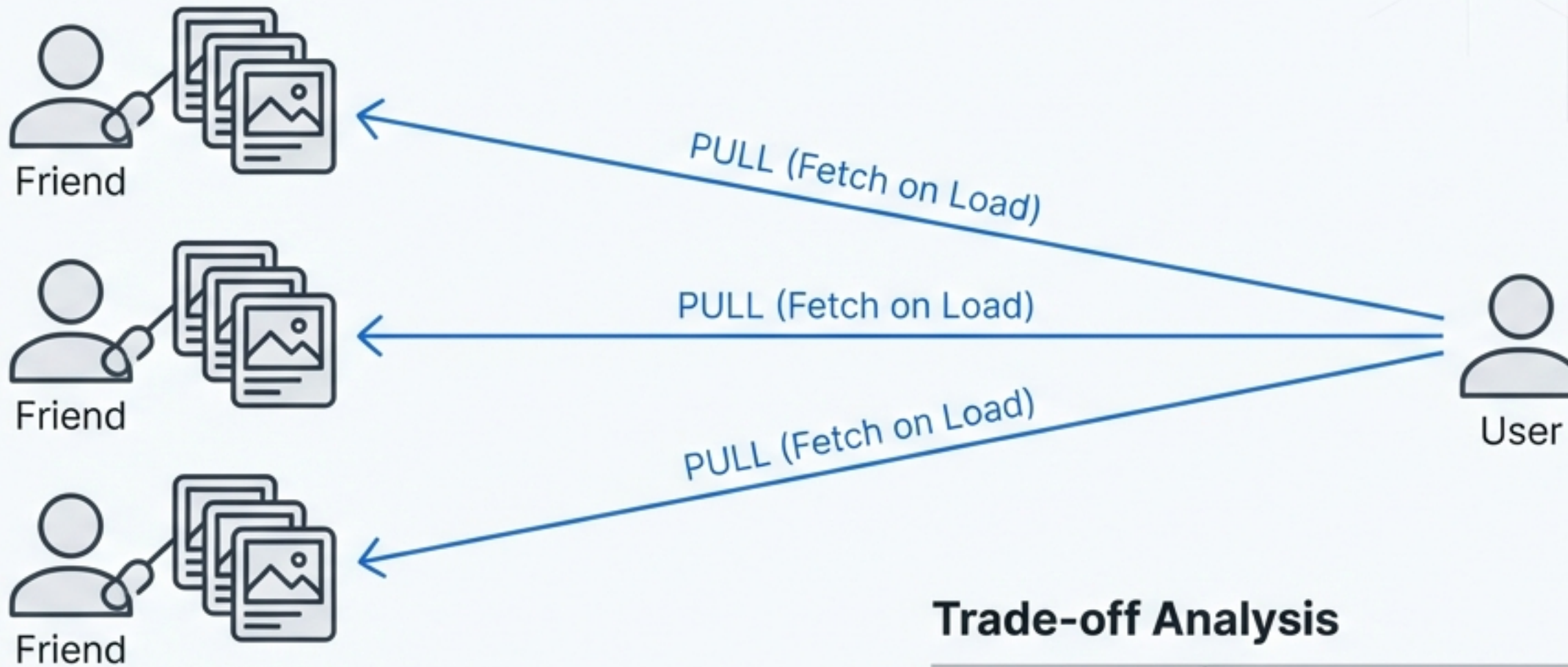
Must handle ever-increasing volume.



FAULT TOLERANCE

Prioritize Availability over strict Consistency.

STRATEGY 1: FAN-OUT-ON-READ (THE PULL MODEL)



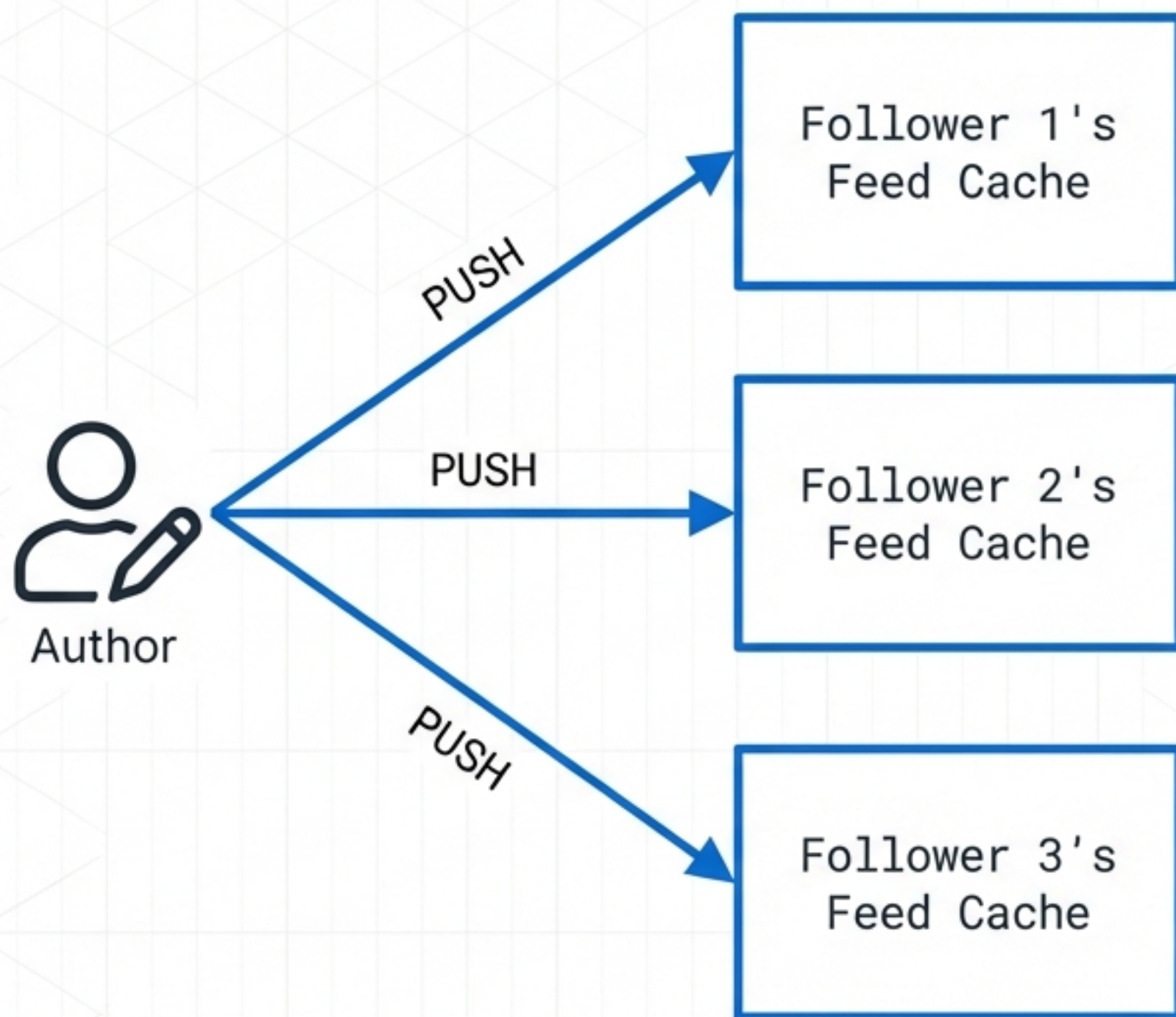
MECHANISM: Posts are stored only in the author's timeline. No follower feeds are updated at write time.

When a user opens the app, the system fetches posts from all followed users/pages and merges/ranks them dynamically.

Trade-off Analysis

PROS	Storage Efficient (No data duplication). No Write Amplification.
CONS ⚠️	High Read Latency (Expensive generation at load time).
BEST FOR	Celebrities (e.g., LeBron James), High-fanout pages, Inactive users.

STRATEGY 2: FAN-OUT-ON-WRITE (THE PUSH MODEL)



MECHANISM: Helvetica Now Display. As soon as a post is created, the system fans out and updates the feed cache of every single follower. The feed is pre-materialized; it is waiting in the cache before the user even opens the app.

Trade-off Analysis

PROS

Blueprint Blue

Extremely Fast Read Latency.
Instant UX.

CONS

Alert Orange

Severe Write Amplification.
(1 Post = Millions of Writes
for celebs).

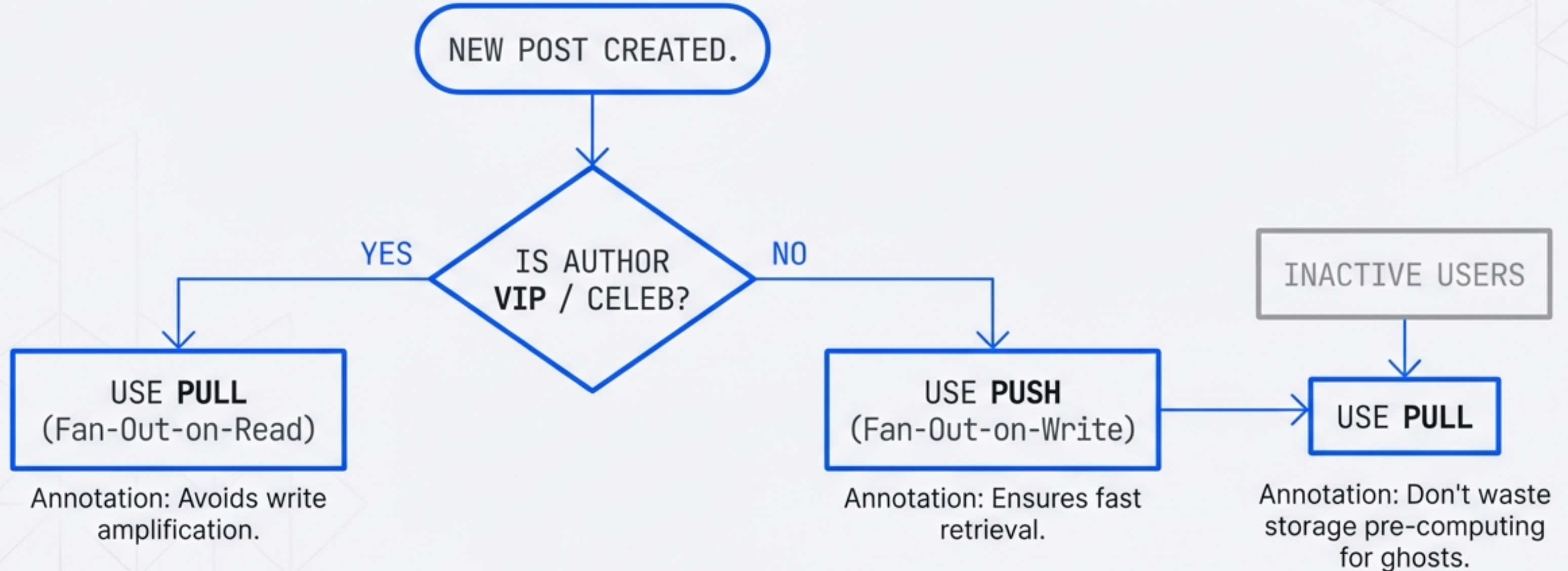
BEST FOR

Slate Grey

Normal users, Close friend
connections, Active users.

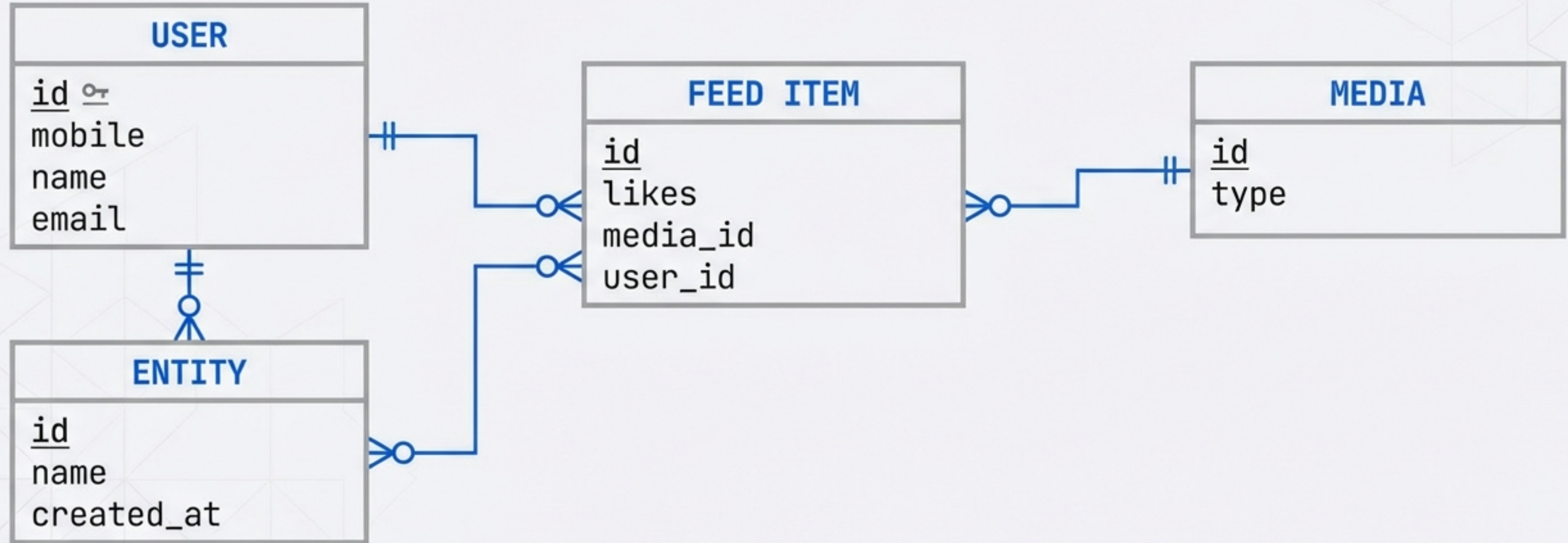
STRATEGY 3: THE HYBRID ARCHITECTURE

Real-world systems combine both approaches to solve the scaling limits of Push and the latency issues of Pull. We optimize based on the user's graph size and activity level.



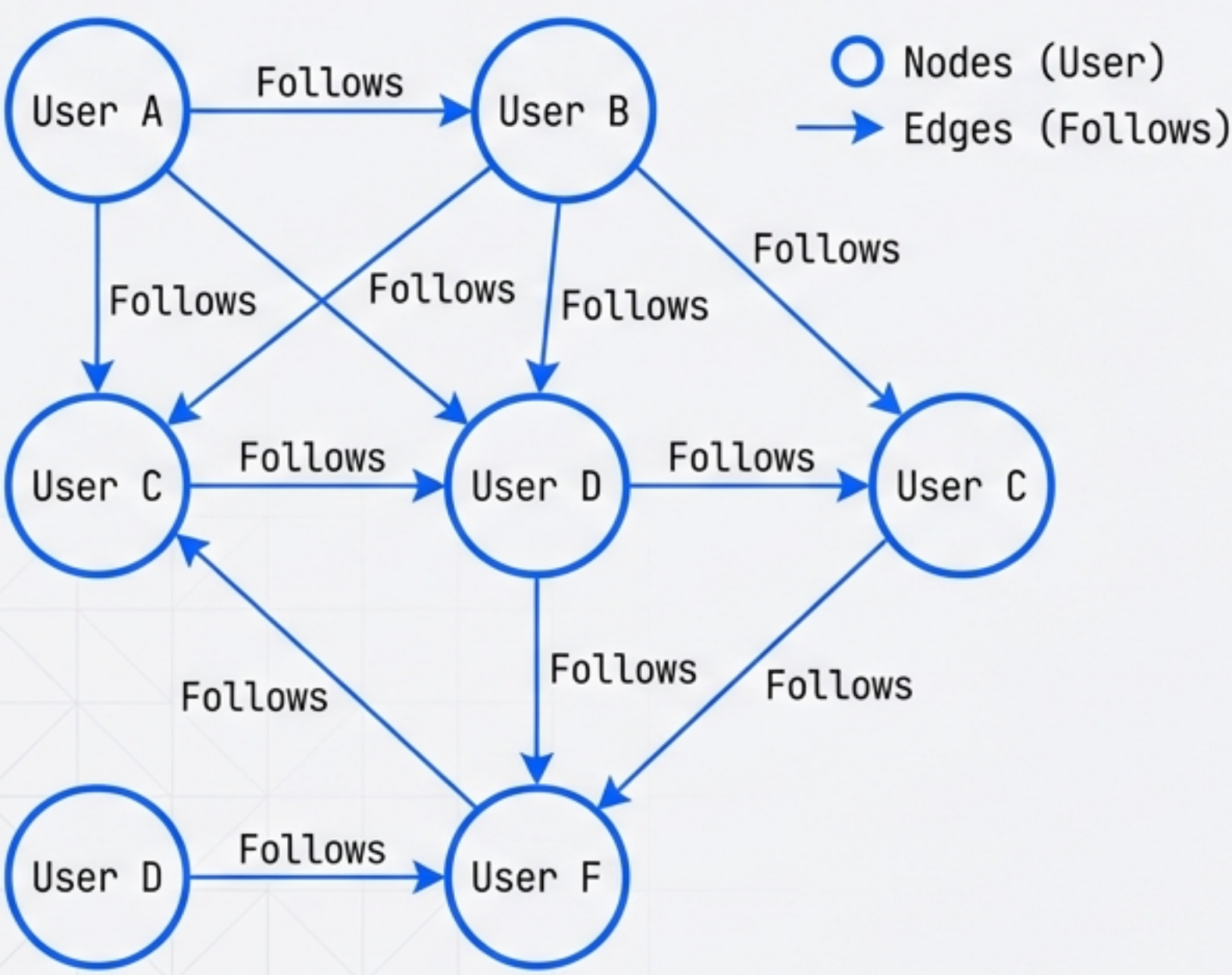
STRUCTURED DATA LAYER: THE RELATIONAL SCHEMA

While relationships are complex, the core entity data is structured and predictable, making SQL the correct choice for User and Feed Item data.



MANAGING CONNECTIONS: THE GRAPH DATABASE

Graph DB is the natural choice for mapping complex relationships (Users, Friends, Followers). It allows for efficient traversal queries (e.g., 'friend of a friend'). It functions using two virtual tables: Vertices and Edges.

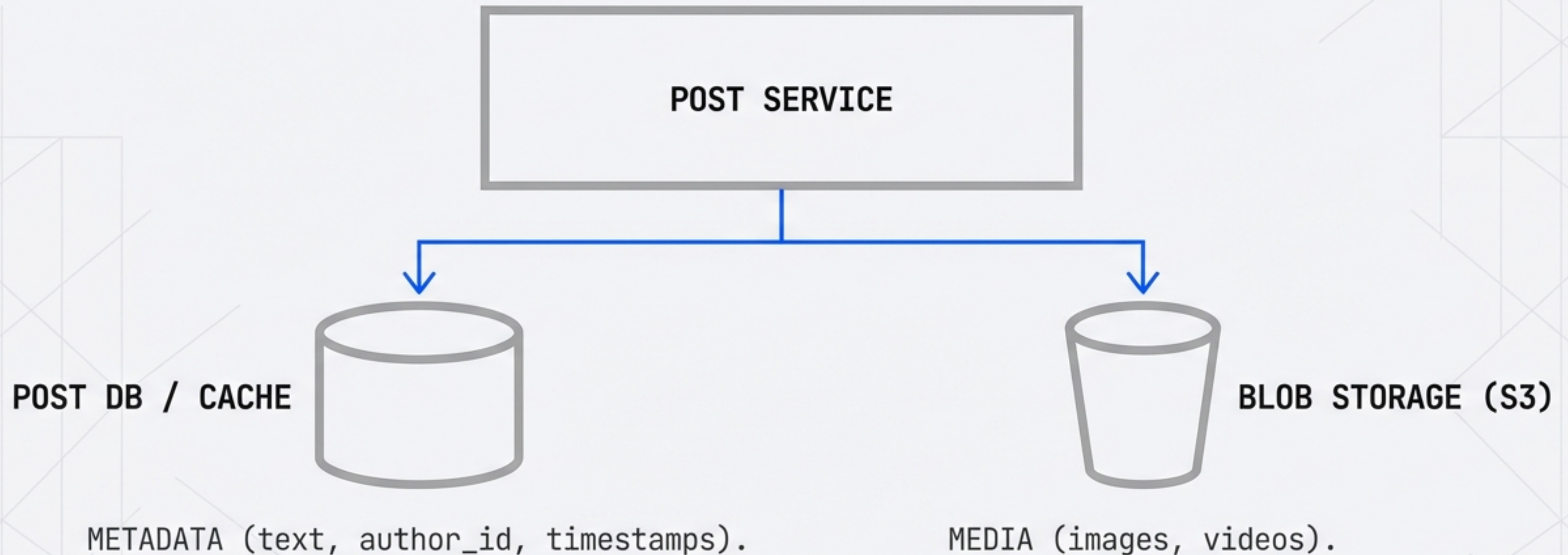


USER (VERTICES)	
user_id: int	properties: json
123	{name: "Alice", location: "NY"}
321	{name: "Bob", location: "SF"}
456	{name: "Charlie", location: "LA"}

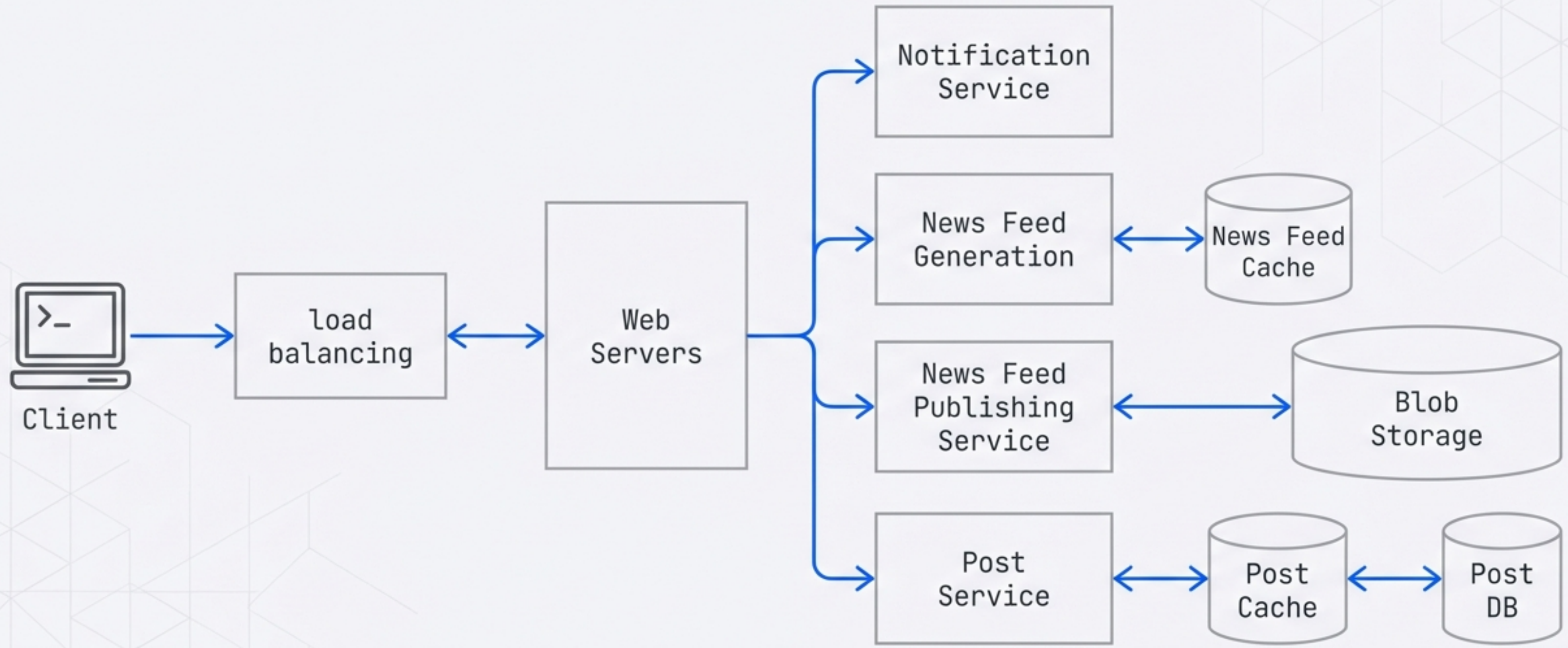
RELATIONSHIP (EDGES)			
relation_id:	relation_from	relation_towards	label
1	123	321	following
2	321	456	following
3	123	456	following

STORAGE INFRASTRUCTURE & BLOB STORAGE SEPARATION

Separation of Concerns: The system splits a post into two distinct data streams. The DB only holds lightweight references (``media_id``) to the heavy objects stored in Blob Storage. This keeps the database performant.

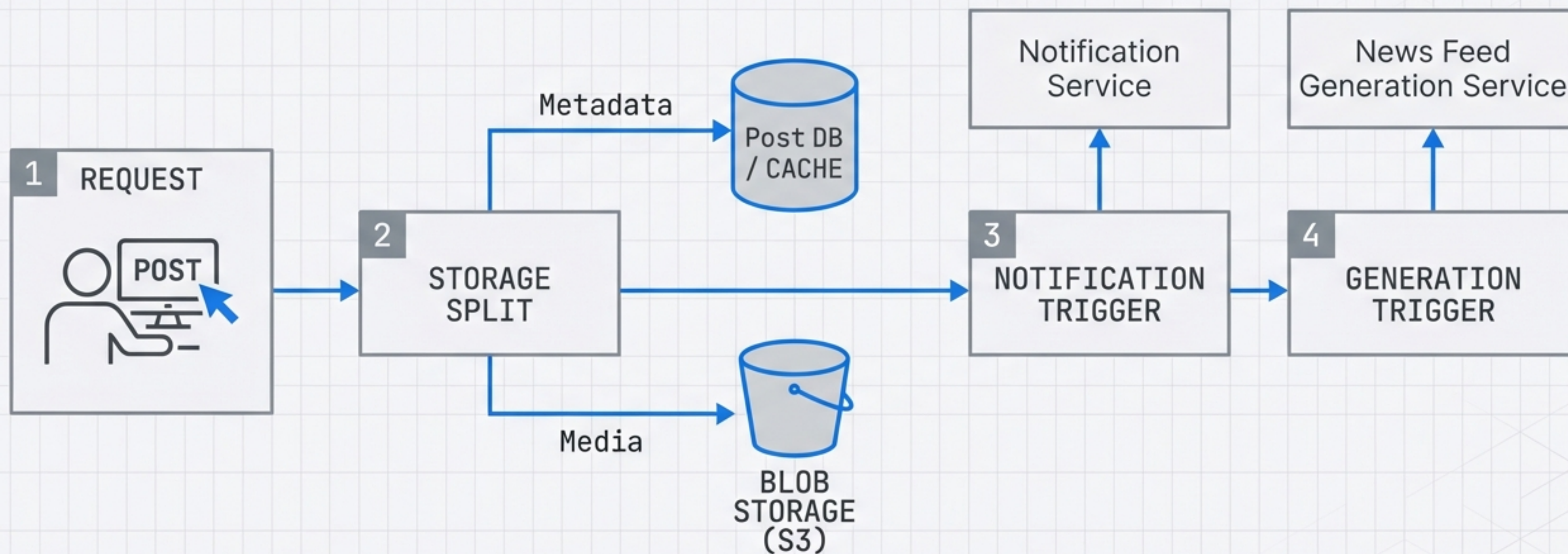


THE HIGH-LEVEL BASE ARCHITECTURE



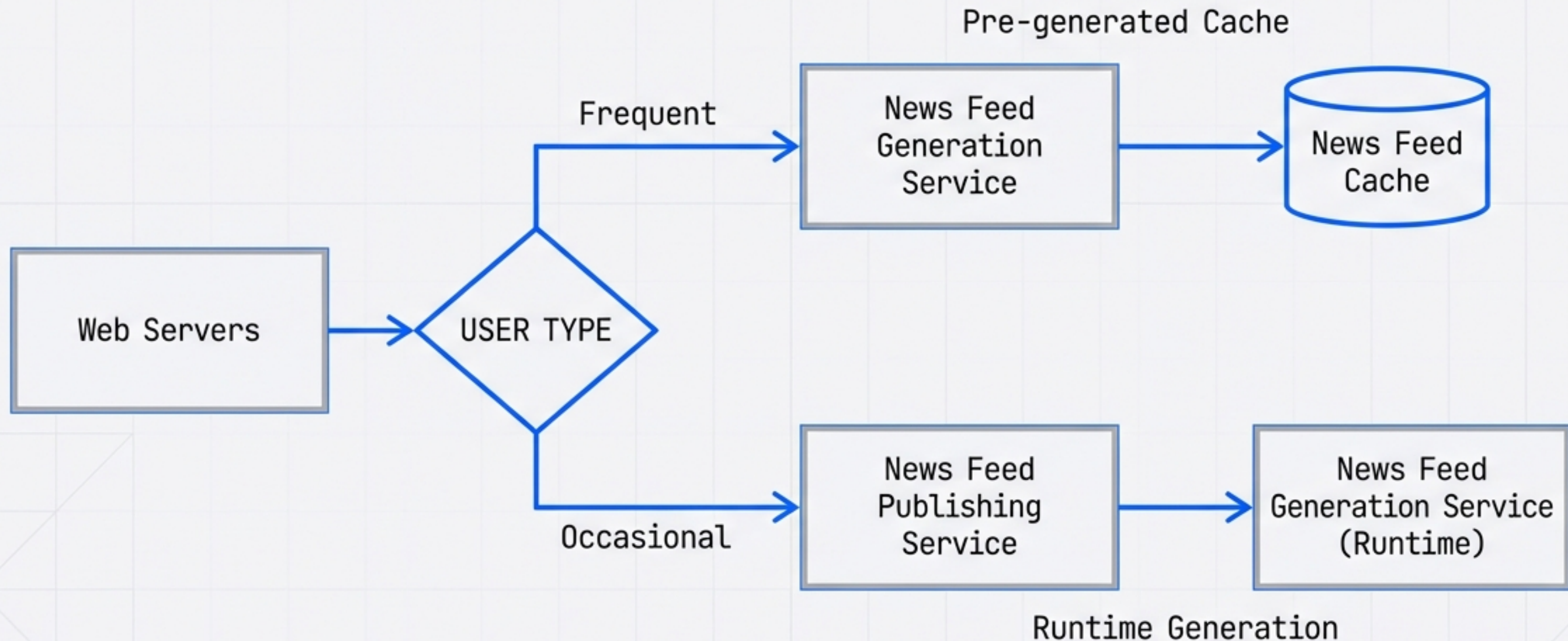
'THE WRITE PATH: POST SERVICE LIFECYCLE'

When a post is created, the service orchestrates storage, notifies followers, and signals the generation service to update feeds.



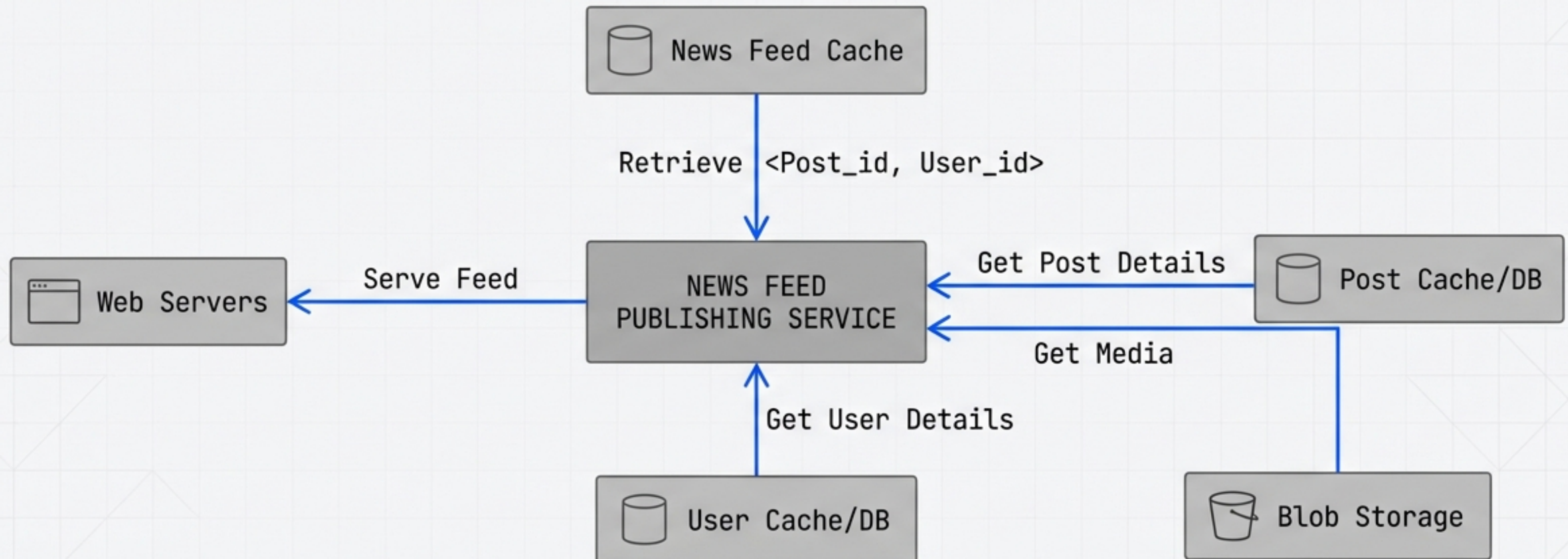
FEED GENERATION LOGIC: FREQUENT VS. OCCASIONAL

Optimization Strategy: We pre-generate feeds for active users to ensure seamless loading. For occasional users (cold), we generate the feed at runtime to save storage.



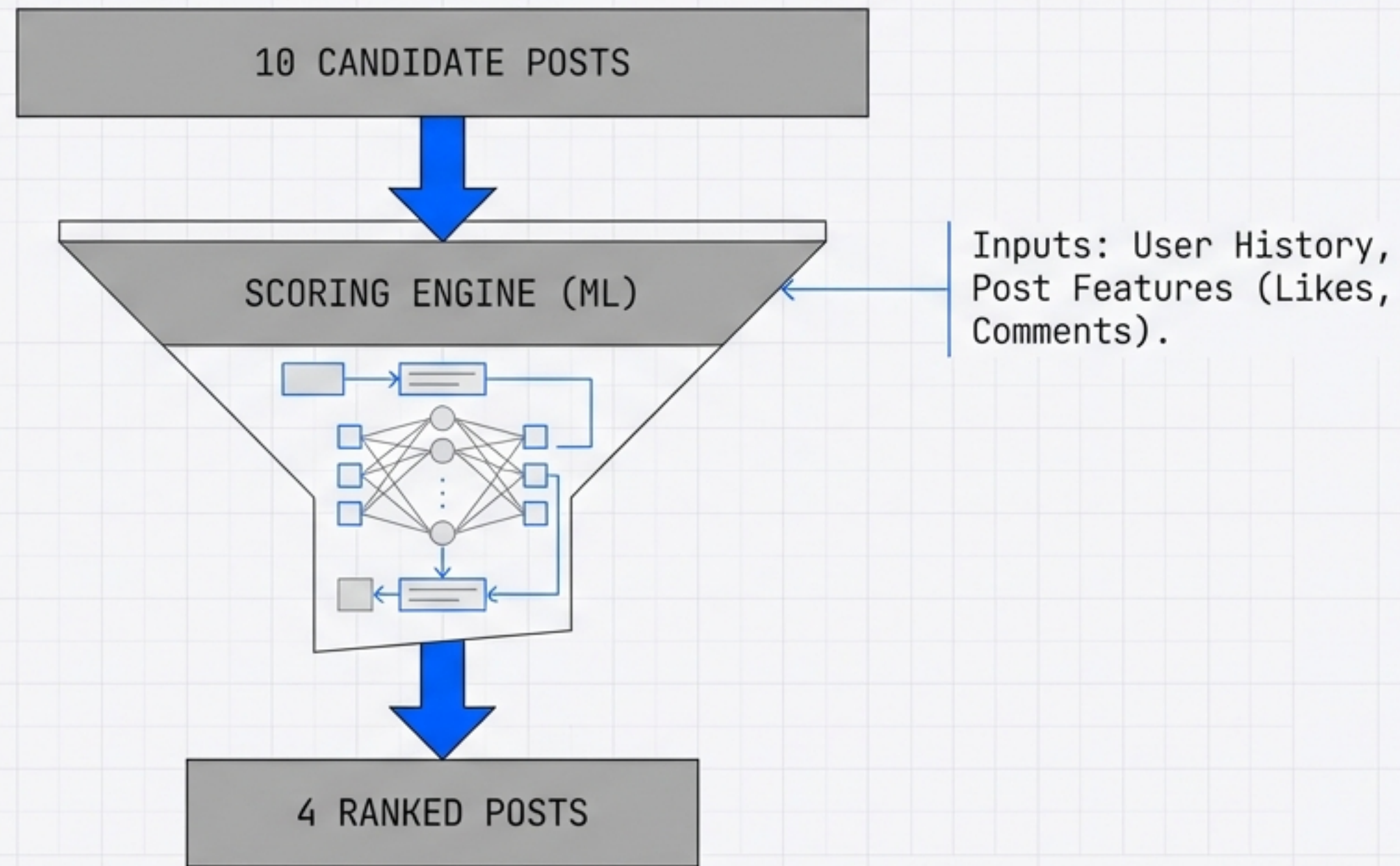
THE READ PATH: NEWS FEED PUBLISHING SERVICE

Hydration Logic: The cache only provides IDs. The publishing service must "hydrate" the feed by fetching actual content (text, user info, media links) from their respective sources before serving to the client.



THE RANKING SERVICE: RELEVANCE OPTIMIZATION

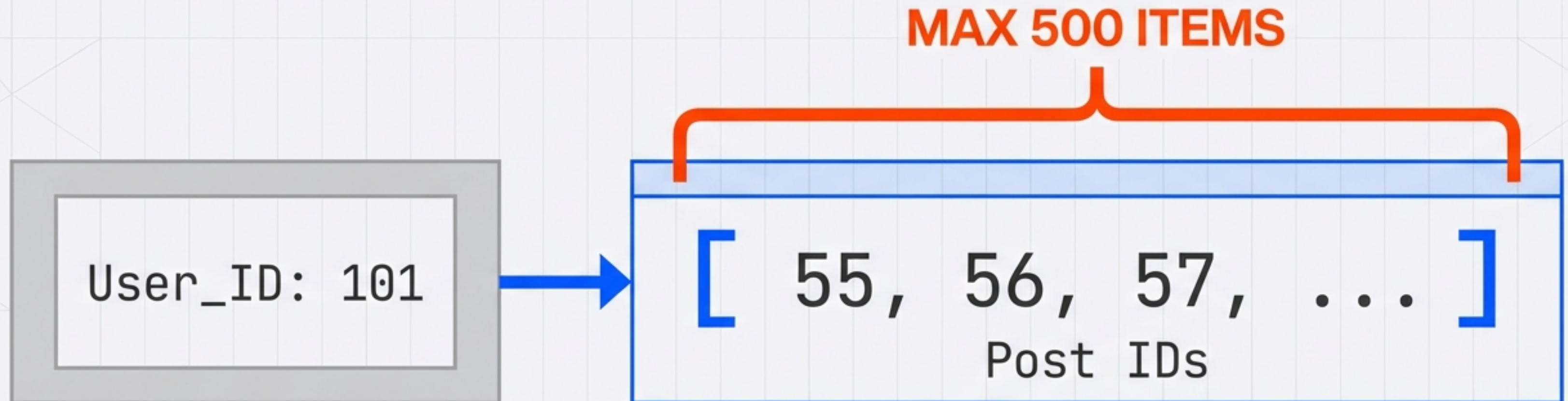
Algorithm: Features + History $\rightarrow$ Relevance Score. If User "Tom" follows 5 users who publish 10 posts, the system scores them based on Tom's past behavior and displays the top 4 in decreasing order of relevance.



CACHING ARCHITECTURE: STORING IDS, NOT CONTENT

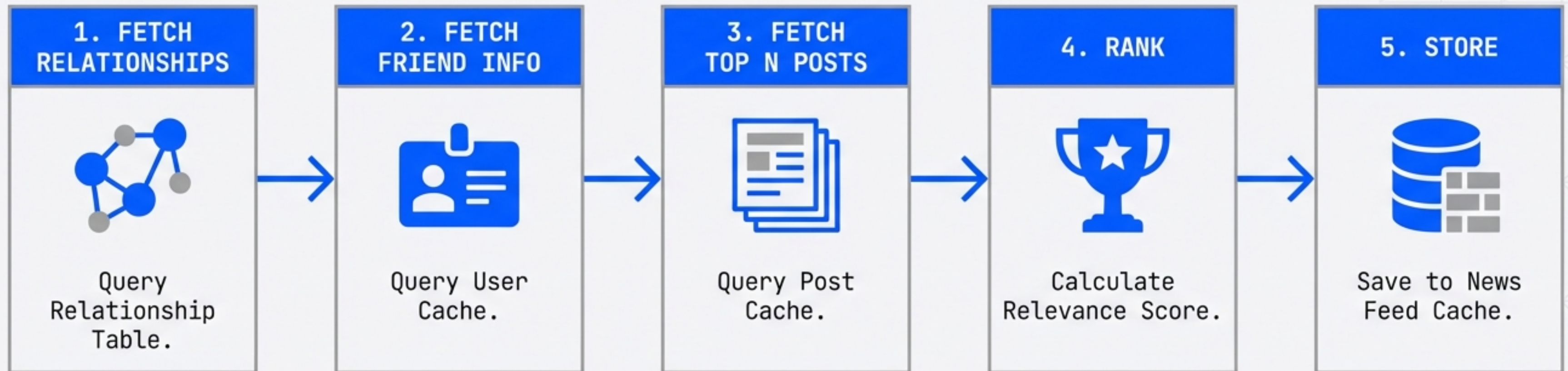
Efficiency: We do not store full JSON objects in the News Feed Cache. We store only the mapping of User IDs to Post IDs.

Constraint: To prevent memory bottlenecks, we cap the cache at 500 posts per user. Old IDs are evicted as new ones arrive.



RUNTIME GENERATION: THE “OCCASIONAL USER” WORKFLOW

Because we don't pre-compute feeds for inactive users, we must perform this heavy lifting at runtime when they open the app.



TRAFFIC & SCALE ESTIMATIONS

1 BILLION USERS

500 MILLION DAU

10 SESSIONS / DAY

$\text{500M Users} * \text{10 Requests} = \text{5 Billion Daily Requests}$

$\text{5 Billion} / \text{86400 Seconds} =$

60,000 REQUESTS PER SECOND (RPS)

STORAGE CAPACITY PLANNING

To keep feeds in-memory for immediate publishing, we require massive RAM capacity distributed across thousands of cache servers.

5000 SERVERS REQUIRED



Metadata Size:
1KB

Cache Per User:
 $500 \text{ items} * 1\text{KB} = 500\text{KB}$

Total Cache:
 $500\text{M DAU} * 500\text{KB} = 500\text{TB RAM}$

SYSTEM DESIGN SUMMARY & GLOSSARY

GOLDEN RULE

Availability > Correctness.

FAN-OUT-ON-WRITE (PUSH)

Fast read, slow write.
Good for normal users.

FAN-OUT-ON-READ (PULL)

Slow read, fast write.
Good for celebs/cold users.

SKELETON LOADING

UI pattern to mask 2-second latency.

GRAPH DB

Optimized for Edges (relationships) and Vertices (Users).

HYBRID STRATEGY

The combination of Push and Pull to optimize scale.

FEED CACHE

Stores IDs only, max 500 per user.